

"""

bitunix_bot_v8.py - v5.1(Kimi audit) + risk-sizing fix + leverage-veto fix (see V

Multi-pair ignition research (Jun-Nov 2025, incl. crash, fees in):

SOL	45.7% WR, +0.50R/trade, 1.8 tr/mo	-> KEEP (1.5% risk)
ETH	40.8% WR, +0.32R/trade, 4.8 tr/mo	-> KEEP (1.5% risk)
BNB	36.4% WR, +0.17R/trade, 4.1 tr/mo	-> WATCH (half risk)
BTC	33.3% WR, +0.05R/trade	-> donchian only, minimal risk
DOGE	26.1% WR, -0.19R/trade	-> CUT
XRP	26.7% WR, -0.17R/trade	-> CUT

Portfolio estimate: ~10-11 trades/mo, +2 to +3%/mo optimistic, ~+2%/mo realistic.
29% of trades cluster within 12h -> total open risk capped at 3%.

LAUNCH: dry-run 1 week -> verify #VERIFY paths -> min size until kill switch
has 40 trades (SOL/ETH) - it judges whether the edge is real.

V8 CHANGES:

1. RISK SIZING: risk_map was left at the original 3.5%/1.75%/1% "max survivable aggression" ceiling values while risk_per_trade (the documented, intended de says 1.5%. RISK_SCALE env (default 0.5) now scales risk_map/scanner_risk/max down to roughly that intended level. Set RISK_SCALE=1 to restore full 3.5% s only do that once you've watched it run at half size and are deliberately ch the larger drawdowns that come with it.
2. LEVERAGE VETO BUG: the RuntimeError raised when set_leverage() failed was be caught by its own enclosing try/except one line below - so "never trade with confirmed leverage" never actually fired; it silently logged a warning and t anyway. Restructured so the veto can actually propagate.
3. LEVERAGE ENDPOINT: none of the 4 paths set_leverage() tried match Bitunix's endpoint (POST /api/v1/futures/account/change_leverage, per their OpenAPI do so leverage was never actually being set; fixed to the real endpoint.

"""

```
import os, time, json, hashlib, logging, csv
from dataclasses import dataclass, field
from datetime import datetime, timezone
import requests
import threading
import math
```

```
BASE = "https://fapi.bitunix.com"
INTERVAL_MS = {"1h": 3600e3, "4h": 14400e3}
```

```
@dataclass
class Config:
```

```

symbols: list = field(default_factory=lambda: ["ETHUSDT", "SOLUSDT", "BNBUSDT",
signal_map: dict = field(default_factory=lambda: {"ETHUSDT": ["ignition", "do
                                                    "SOLUSDT": ["ignition", "do
                                                    "BNBUSDT": ["ignition"],
                                                    "BTCUSDT": ["donchian"],
                                                    "DOTUSDT": ["donchian"],
                                                    "NEARUSDT": ["ignition"]})
risk_map: dict = field(default_factory=lambda: {"ETHUSDT": 0.035, "SOLUSDT":
                                                    "BNBUSDT": 0.0175, "BTCUSDT
                                                    "DOTUSDT": 0.0175, "NEARUSD

max_open_risk: float = 0.07
min_qty_map: dict = field(default_factory=lambda: {"BTCUSDT": 0.001})
min_notional_usdt: float = 15.0
mode: str = field(default_factory=lambda: os.getenv("MODE", "strict").lower())
#   strict = validated gates (positive in ALL tested regimes - recommended)
#   active = looser gates (~40% more trades, regime-dependent edge - your cal
surge_mult: float = field(default_factory=lambda: float(os.getenv("SURGE_MULT
trigger_dist: float = field(default_factory=lambda: float(os.getenv("TRIGGER_
scanner_dynamic: bool = True      # build universe from ALL Bitunix perps (vol
scanner_min_vol24: float = 2_000_000.0  # 24h volume floor (Bitunix-native v
scanner_max_pairs: int = 40          # cap on universe size per scan
scanner_pairs: list = field(default_factory=lambda: ["BTCUSDT", "ETHUSDT", "SOL
                                                    "DOGEUSDT", "LINKUSDT", "
                                                    "1000PEPEUSDT", "WIFUSDT

scanner_risk: float = 0.0125        # per top-gainer trade (validated two window
scanner_max: int = 2                # max concurrent scanner positions
scanner_ret: float = 0.04           # min 24-bar gain to qualify
entry_limit: bool = False           # market entries on breakouts (limit entries
limit_offset: float = 0.0005        # place limit 0.05% better than signal price
entry_timeoutBars: int = 2          # bars to wait for fill, then cancel
interval: str = "4h"
donchian_n: int = 20
ema_trend: int = 200
risk_per_trade: float = 0.015        # 1% - chassis validated for this struct
stop_atr: float = 1.0               # stop = 1x ATR
tp_r: float = 2.5                   # validated 2.0-2.5 range
stop_min: float = 0.008
stop_max: float = 0.05
max_notional_lev: float = 10.0
vol_spike: float = 1.5
flow_long: float = 0.55
flow_short: float = 0.45
daily_loss_limit: float = field(default_factory=lambda: float(os.getenv("DAIL
kill_min_trades: int = 40           # lower: candidate edge needs faster verd
kill_wr_buffer: float = 0.02
poll_seconds: int = field(default_factory=lambda: int(os.getenv("POLL_SECONDS
dry_run: bool = True

```

```

dry_equity: float = 10_000.0
tg_token: str = field(default_factory=lambda: os.getenv("TG_TOKEN", "")) #
tg_chat: str = field(default_factory=lambda: os.getenv("TG_CHAT", "")) #
risk_scale: float = field(default_factory=lambda: float(os.getenv("RISK_SCALE", "1.0")))
def __post_init__(self):
    s = self.risk_scale
    self.risk_map = {k: v*s for k, v in self.risk_map.items()}
    self.scanner_risk *= s
    self.max_open_risk *= s

FEE_M, FEE_T, SLIP = 0.0002, 0.0006, 0.0002
logging.basicConfig(level=logging.INFO, format="%asctime)s %(levelname)s %(message)s",
                    handlers=[logging.FileHandler("bot.log"), logging.StreamHandler()])

_ACCT_DEBUGGED = False
_TK_DEBUGGED = False
_LAST_EQ = None
_TG_TOKEN = os.getenv("TG_TOKEN", "")
_TG_CHAT = os.getenv("TG_CHAT", "")
def notify(msg):
    """Send msg to your Telegram - the bot's voice on your phone."""
    if not (_TG_TOKEN and _TG_CHAT): return
    try:
        requests.post(f"https://api.telegram.org/bot{_TG_TOKEN}/sendMessage",
                      data={"chat_id": _TG_CHAT, "text": msg, "parse_mode": "HTML",
                            "disable_web_page_preview": True}, timeout=10)
    except Exception:
        pass

def foot():
    return f"\n- LeverageLord · {datetime.now(timezone.utc).strftime('%H:%M')} UT"

def streak_of(trades):
    s = 0
    for t in reversed(trades):
        if t: s += 1
        else: break
    return s

def binance_klines(symbol, interval="4h", limit=100):
    """Public market data from Binance's geo-safe data endpoint (no key, no US bl
    iv = {"1h": "1h", "4h": "4h", "6h": "6h", "1d": "1d"}.get(interval, "4h")
    cand = [symbol] + ([symbol[4:]] if symbol.startswith("1000") else [])
    last_err = "no response"
    for cand in cand:
        try:
            r = requests.get("https://data-api.binance.vision/api/v3/klines",

```

[illegible]

```

        for k, v in sorted(params.items()):
            qp += f"{k}{v}"
        inner = hashlib.sha256((nonce+ts+self.key+qp+body_str).encode()).hexdigest
        return {"api-key": self.key, "nonce": nonce, "timestamp": ts,
                "sign": hashlib.sha256((inner+self.secret).encode()).hexdigest(),
                "language": "en-US", "Content-Type": "application/json"}

def _req(self, method, path, params=None, payload=None):
    body_str = json.dumps(payload, separators=(",", ":")) if payload else ""
    r = self.s.request(method, BASE+path, params=params, data=body_str or None,
                       headers=self._sign(method, path, body_str, params), timeout=10)
    try:
        data = r.json()
    except Exception:
        raise RuntimeError(f"{path} HTTP {r.status_code}: {r.text[:200]}")
    if data.get("code") != 0:
        raise RuntimeError(f"{path} -> code {data.get('code')}: {data.get('message')}")
    return data.get("data")

def klines(self, symbol, interval, limit=200, start_time=None, end_time=None):
    limit = min(limit, 200)
    if start_time is None:
        span = int(limit * INTERVAL_MS.get(interval, 14400e3))
        start_time = int(time.time()*1000) - span - 7200000 # explicit receive time
    params = {"symbol": symbol, "interval": interval, "limit": limit, "startTime": start_time}
    if end_time: params["endTime"] = str(int(end_time))
    rows = self._req("GET", "/api/v1/futures/market/kline", params)
    return [{"o":float(r["open"]), "h":float(r["high"]), "l":float(r["low"]),
            "c":float(r["close"]), "t":int(r["time"]),
            "vol":float(r.get("vol",0)), "tb":float(r.get("takeVol", r.get("takeVol",0)))}

def trading_spec(self, symbol):
    """Exchange-native quantity/price constraints (v5.1 audit improvement)."""
    try:
        data = self._req("GET", "/api/v1/futures/market/trading_pairs", {"symbol": symbol})
        row = data[0] if isinstance(data, list) and data else {}
        return {"min_qty": float(row.get("minTradeVolume") or 0),
                "qty_precision": int(row.get("basePrecision") or 6),
                "px_precision": int(row.get("quotePrecision") or 8),
                "max_market_qty": float(row.get("maxMarketOrderVolume") or 0)}
    except Exception:
        return None

def normalize_qty(self, symbol, qty):
    """Pre-fit qty to exchange rules; ladder remains as the safety net."""
    try:
        sp = self.trading_spec(symbol)
        if not sp: return qty
        step = 10 ** (-max(0, sp["qty_precision"]))
        q = math.floor(float(qty) / step + 1e-12) * step
    except Exception:
        return qty

```

```

        q = max(q, sp["min_qty"] or 0)
        if sp["max_market_qty"] > 0: q = min(q, sp["max_market_qty"])
        return q
    except Exception:
        return qty

def history_position(self, position_id):
    """Realized PnL/funding/fees for a closed position (v5.1 audit improvemen
    try:
        data = self._req("GET", "/api/v1/futures/position/get_history_positio
            {"positionId": str(position_id), "limit": 10}) or {}
        rows = data.get("positionList", []) if isinstance(data, dict) else (d
        for row in rows:
            if str(row.get("positionId")) == str(position_id):
                return row
    except Exception:
        pass
    return None

def flash_close(self, position_id):
    for path in ["/api/v1/futures/trade/flash_close_position",
        "/api/v1/futures/position/flash_close_position"]:
        try:
            return self._req("POST", path, payload={"positionId": str(positio
        except Exception:
            continue
    raise RuntimeError("flash_close failed on all paths")

def set_leverage(self, symbol, leverage):
    errs = []
    for path in ["/api/v1/futures/account/change_leverage"]: # v8 fix: this
                                                                # the other
        try:
            self._req("POST", path, payload={"symbol": symbol, "marginCoin":
                "leverage": str(int(leverage))})
            logging.info(f"leverage set {symbol} {leverage}x")
            return True
        except Exception as e:
            errs.append(f"{path}: {str(e)[:80]}")
    logging.warning(f"set_leverage failed for {symbol} | " + " | ".join(errs)
    return False

def ticker_px(self, symbol):
    now = time.time()
    c = self._px_cache.get(symbol)
    if c and now - c[0] < 20:
        return c[1]

```

```

# v8 fix: this used to read rows[0] assuming the "symbol" filter worked -
# Bitunix returns the full tickers list regardless, so rows[0] was always
# symbol happens to sort first (e.g. ALABUSDT) - NOT the symbol asked for
# silently fed the wrong coin's price into stop/target checks and R calcu
rows = self._req("GET", "/api/v1/futures/market/tickers", {"symbols": sym
row = next((r for r in rows if r.get("symbol") == symbol), None)
if row is None:
    rows = self._req("GET", "/api/v1/futures/market/tickers") or []
    row = next((r for r in rows if r.get("symbol") == symbol), None)
px = float(row.get("lastPrice") or 0) if row else 0.0
self._px_cache[symbol] = (now, px)
return px

def tickers(self):    # Bitunix 'Get All Tickers' (confirmed path)
    rows = self._req("GET", "/api/v1/futures/market/tickers") or []
    global _TK_DEBUGGED
    if not _TK_DEBUGGED and rows:
        _TK_DEBUGGED = True
        logging.info(f"TICKERS RAW sample: {json.dumps(rows[0])[:400]} | coun
    out = []
    for t in rows:
        sym = t.get("symbol", "")
        vol = float(t.get("vol") or t.get("volume") or t.get("vol24h") or t.g
        out.append({"symbol": sym, "vol24": vol,
                    "lastPrice": float(t.get("lastPrice") or 0),
                    "markPrice": float(t.get("markPrice") or 0),
                    "open": float(t.get("open") or 0)})
    return out

def valid_symbols(self):
    try:
        syms = {t["symbol"] for t in self.tickers() if t.get("symbol")}
        if syms: logging.info(f"valid futures symbols: {len(syms)} listed")
        return syms
    except Exception as e:
        logging.warning(f"symbol discovery failed: {e} - allowing configured
        return None

def equity_usdt(self):
    global _LAST_EQ
    acct = None; used = "?"; errs = []
    for path in ["/api/v1/futures/account",                      # CONFIRMED via Bitu
                 "/api/v1/futures/account/assets"]:
        try:
            acct = self._req("GET", path, {"marginCoin": "USDT"})
            if acct is not None:
                used = path; break

```

```

        except Exception as e:
            errs.append(f"{path} => {e}")
    if acct is None:
        logging.warning("ACCOUNT FAIL | " + " | " + ".join(errs)[:500])
    if _LAST_EQ is not None:
        logging.info(f"using cached equity {_LAST_EQ}")
        return _LAST_EQ
    raise RuntimeError("account unreachable: " + " " + ".join(errs)[:250])
a = acct[0] if isinstance(acct, list) and acct else (acct if isinstance(a
global _ACCT_DEBUGGED
if not _ACCT_DEBUGGED:
    _ACCT_DEBUGGED = True
    logging.info(f"ACCOUNT RAW via {used}: {json.dumps(a)[:300]}")
def f(*keys):
    for k in keys:
        if k in a and a[k] not in (None, ""):
            return float(a[k])
    return 0.0
eq = f("available", "availableVol", "availableBalance") + f("margin", "posit
_LAST_EQ = eq
return eq

def positions(self, symbol=None):
    params = {"symbol": symbol} if symbol else {}
    rows = None
    for path in ["/api/v1/futures/position/get_pending_positions", # confir
                 "/api/v1/futures/position/pending_positions"]:
        try:
            rows = self._req("GET", path, params)
            if rows is not None: break
        except Exception:
            continue
    rows = rows or []
    out=[]
    for p in rows:
        q=float(p.get("qty") or p.get("size") or p.get("positionSize") or 0)
        if q>0: out.append({"id":str(p.get("positionId")), "sym":p["symbol"],
                           "side":1 if p.get("side")== "LONG" else -1, "qty":
    return out
def cancel(self, symbol, order_id):
    for path, payload in [
        ("/api/v1/futures/trade/cancel_orders", {"symbol": symbol, "order
        ("/api/v1/futures/trade/cancel_order", {"symbol": symbol, "orderI
        ("/api/v1/futures/order/cancel_order", {"symbol": symbol, "orderI
    try:
        return self._req("POST", path, payload=payload)
    except Exception:

```

```

        continue
def place(self, symbol, side, qty, trade_side, order_type="MARKET", price=None,
        stop=None, target=None, tick=0.1, position_id=None):
    # returns ("bracket"|"plain", result).
    # OPEN orders: auto-retry qty at every valid precision (0.001 -> 1.0), then
    if trade_side == "OPEN":
        steps = [0.001, 0.01, 0.1, 1.0]
        last_e = None
        for st in steps:
            q2 = math.floor(qty/st)*st
            if q2 <= 0: q2 = st
            try:
                return ("bracket", self._place(symbol, side, q2, trade_side,
                except Exception as e:
                    last_e = e
                    if "LEVERAGE VETO" in str(e):
                        raise # never trade without confirmed leverage
                    if "10002" in str(e):
                        continue # wrong qty precision -> need more precision
                    logging.warning(f"bracket rejected ({e}) - trying plain order")
                    try:
                        return ("plain", self._place(symbol, side, q2, trade_side,
                    except Exception as e2:
                        last_e = e2
                        if "10002" in str(e2):
                            continue
                        raise
            raise last_e
        try:
            return ("bracket", self._place(symbol, side, qty, trade_side, order_type,
        except Exception as e:
            if "LEVERAGE VETO" in str(e):
                raise
            if "place_order" in str(e):
                logging.warning(f"bracket rejected ({e}) - placing plain order, b
                return ("plain", self._place(symbol, side, qty, trade_side, order
            raise

def _place(self, symbol, side, qty, trade_side, order_type="MARKET", price=None,
        stop=None, target=None, tick=0.1, position_id=None):
    payload={"symbol":symbol,"side":side,"qty":fmt_qty(qty),"tradeSide":trade_side,
            "orderType":order_type,"effect":"GTC","clientId":str(int(time.time()*1000000000)),
            "reduceOnly": trade_side=="CLOSE"}
    if position_id:
        payload["positionId"] = str(position_id)
    if order_type=="LIMIT": payload["price"]=fmt_px(price,tick)
    if trade_side == "OPEN" and stop and target:

```

```

atr = abs(target - stop) / 3.5
entry_est = (stop + atr) if side == "BUY" else (stop - atr)
stop_pct = atr / max(entry_est, 1e-9)
lev = max(5, min(15, int(0.9 / (3 * stop_pct))))
if not self.set_leverage(symbol, lev):
    # v8 fix: this used to be caught by a try/except one line below i
    # never actually blocked anything - now it does what the message
    raise RuntimeError(f"LEVERAGE VETO: could not set {lev}x on {symb
if trade_side=="OPEN" and stop and target:
    try: # clamp bracket against a FRESH mark price - fast markets can
        mk = self.klines(symbol, "4h", 1)[-1]["c"] # client has no cfg
        if side == "BUY":
            target = max(target, mk*1.002); stop = min(stop, mk*0.998)
        else:
            target = min(target, mk*0.998); stop = max(stop, mk*1.002)
    except Exception:
        pass
if trade_side=="OPEN" and stop and target:
    payload.update({"tpPrice":fmt_px(target,tick),"tpStopType":"MARK_PRIC
                    "slPrice":fmt_px(stop,tick),"slStopType":"MARK_PRICE"
logging.info(f"ORDER payload: {payload}")
return self._req("POST", "/api/v1/futures/trade/place_order", payload=pay

```

```

class RiskManager:
    def __init__(self, cfg):
        self.cfg=cfg; self.day=datetime.now(timezone.utc).date()
        self.day_start_equity=None; self.trades=[]; self.halted_today=False; self
    def daily_loss_hit(self, equity):
        today=datetime.now(timezone.utc).date()
        if today!=self.day: self.day,self.halted_today=today,False; self.day_star
        self.day_start_equity=self.day_start_equity or equity
        if (not self.halted_today and self.day_start_equity and equity>0
            and equity<=self.day_start_equity*(1-self.cfg.daily_loss_limit)):
            self.halted_today=True; logging.warning("CIRCUIT BREAKER: daily loss
        return self.halted_today
    def record_trade(self, pnl): self.trades.append(pnl>0)
    def kill_switch(self):
        if self.killed or len(self.trades)<self.cfg.kill_min_trades: return False
        wr=sum(self.trades[-self.cfg.kill_min_trades:])/self.cfg.kill_min_trades
        # 2.5:1 payoff break-even at 1% risk incl. fees ≈ 30.5%
        be=0.305
        if wr < be+self.cfg.kill_wr_buffer:
            self.killed=True; logging.error(f"KILL SWITCH: WR {wr:.1%} < {be:.1%}
        return self.killed

```

```

class Bot:
    def __init__(self, cfg, client):

```

```

self.cfg, self.client = cfg, client
self.risk = RiskManager(cfg)
self.state = {s: 0 for s in cfg.symbols}
self.seen_bar = {s: 0 for s in cfg.symbols}
self.open_pos = {}; self.entry_info = {}
self.scan_seen_bar = 0
self.tg_offset = 0
self.triggers = [] # [{sym, side, level, stop, target, qty, expiry}]
self.last_scan = {"top": [], "universe": 0, "signals": 0}
self.scan_data = {}
self.eq_hist = []
self.day_stamp = datetime.now(timezone.utc).strftime("%Y-%m-%d")
self.day_start_eq = None
self.csv = open("trades.csv", "a", newline="")
self.writer = csv.writer(self.csv)
if self.csv.tell() == 0:
    self.writer.writerow(["time", "symbol", "signal", "side", "qty", "stop", "t
def log(self, *row):
    self.writer.writerow([datetime.now(timezone.utc).isoformat()] + list(row));
def sync(self):
    live = {p["id"]: p for p in self.client.positions()}
    for pid, info in list(self.open_pos.items()):
        if pid not in live:
            R = None
            if info.get("eq_at_entry") and info.get("risk_frac"):
                try:
                    hp = self.client.history_position(pid)
                    if hp:
                        net = float(hp.get("realizedPNL") or 0) + float(hp.ge
                        rd_ = float(info["eq_at_entry"]) * float(info["risk_f
                        if rd_ > 0: R = net / rd_
                except Exception as e:
                    logging.warning(f"history lookup failed for {pid}: {e}")
            if R is None and info.get("entry") and info.get("stop"):
                try:
                    mk = self.client.ticker_px(info["sym"]) or self.client.kl
                    if abs(mk/info["entry"] - 1) < 0.25: # reject corrupt
                        R = (mk - info["entry"]) / abs(info["entry"] - info["stop"])
                except Exception:
                    R = None
            if not (info.get("entry") and info.get("stop")):
                self.log(info["sym"], "CLOSED", "", "", "", "", "", "", "external clo
                notify(f"🟡 <b>{info['sym']} closed externally</b> - not coun
                self.state[info["sym"]] = 0; self.entry_info.pop(info["sym"],
            if R is None: R = 1.0 if info["pnl"] > 0 else -1.0
            self._mr_cool = getattr(self, "_mr_cool", {})
            self._mr_cool[info["sym"]] = time.time()

```

```

R = max(-3.0, min(3.5, R)) # sanity clamp: corrupt dat
if info.get("kind") != "test":
    self.risk.record_trade(R>0)
wins = sum(self.risk.trades); n = len(self.risk.trades)
wr = f"{wins}/{n} ({wins/n*100:.0f}%" if n else "0/0"
e = "🔪" if info.get("kind")=="test" else ("🟢" if R>0 else "🔴")
w = "TEST CLOSED" if info.get("kind")=="test" else ("WIN ✅" if R
pct = R * info.get("risk_frac", 0.015) * 100
rf = info.get("risk_frac", 0.015); eq0 = info.get("eq_at_entry",
eq_now = eq0*(1+R*rf) if eq0 else 0
st = streak_of(self.risk.trades)
self.log(info["sym"], "CLOSED", "", "", "", "", "", f"{R:+.2f}R ({pct:+.
eqline = f"\n🏠 Equity ${eq_now:.2f}" if eq_now else ""
stline = f"🔥{st} win streak" if st >= 2 else ""
notify(f"{e} <b>{info['sym']} CLOSED · {w}</b>\n"
    f"💰 Result: <b>{pct:+.2f}%</b> ({R:+.2f}R)\n🏆 Record: {v
    self.state[info["sym"]]=0; self.entry_info.pop(info["sym"],None);
for pid,p in live.items():
    if pid not in self.open_pos:
        self.open_pos[pid]={"sym":p["sym"],"pnl":0.0}; self.state[p["sym"]
        logging.warning(f"RECONCILE: adopted position {pid} {p['sym']}");
def equity(self):
    return self.cfg.dry_equity if self.cfg.dry_run else self.client.equity_us

def _ema(self, vals, n):
    k=2/(n+1); out=[vals[0]]
    for v in vals[1:]: out.append(v*k+out[-1]*(1-k))
    return out

def open_risk(self):
    """Risk committed across ALL engines, from live tracked positions."""
    return sum(i.get("risk_frac", 0) for i in self.open_pos.values())

def detect_signal(self, sym, rows):
    """Returns (side, kind, atr) or (0,None,0). Signals: donchian / ignition
    closes=[r["c"] for r in rows]
    e200=self._ema(closes, self.cfg.ema_trend)[-1]
    n=self.cfg.donchian_n
    hi=max(r["h"] for r in rows[-n-1:-1]); lo=min(r["l"] for r in rows[-n-1:-
    price=closes[-1]
    # ATR(14)
    trs=[max(rows[i]["h"]-rows[i]["l"], abs(rows[i]["h"]-rows[i-1]["c"]), abs
        for i in range(1,len(rows)))
    atrv=sum(trs[-14:])/14
    br = (rows[-1]["tb"]/rows[-1]["vol"]) if rows[-1]["vol"]>0 else 0.5
    allowed = self.cfg.signal_map.get(sym, ["donchian"])
    for kind in allowed:

```

```

sig=0
if kind=="donchian":
    sig = 1 if (price>hi and price>e200) else -1 if (price<lo and pri
    # order-flow confirm
    if sig>0 and br<self.cfg.flow_long: sig=0
    if sig<0 and br>self.cfg.flow_short: sig=0
elif kind=="ignition":
    c3=closes[-4:]
    vols=[r["vol"] for r in rows[-22:-2]]
    vma=sum(vols)/len(vols) if vols else 0
    up3 = c3[3]>c3[2]>c3[1]>c3[0]
    dn3 = c3[3]<c3[2]<c3[1]<c3[0]
    if up3 and vma>0 and rows[-1]["vol"]>self.cfg.vol_spike*vma and p
    elif dn3 and vma>0 and rows[-1]["vol"]>self.cfg.vol_spike*vma and
    if sig!=0:
        return sig, kind, atrv
return 0, None, atrv

def run_symbol(self, sym, equity):
    try:
        rows = binance_klines(sym, self.cfg.interval, self.cfg.ema_trend + 20
    except Exception as e:
        logging.warning(f"BINANCE FEED DOWN ({e}) - falling back to Bitunix k
        rows = self.client.klines(sym, self.cfg.interval, 200)
    if len(rows) < self.cfg.ema_trend+5: return
    bar = rows[-2] # last CLOSED bar
    win = rows[-1]["t"] # forming bar open == si
    if win==self.seen_bar[sym]: return
    self.seen_bar[sym]=win
    sig, kind, atrv = self.detect_signal(sym, rows[:-1])
    if sig == 0:
        try:
            r = rows[:-1]
            br = (r[-1]["tb"]/r[-1]["vol"]) if r[-1]["vol"]>0 else 0.5
            n = self.cfg.donchian_n
            hi = max(x["h"] for x in r[-n-1:-1]); lo = min(x["l"] for x in r[
            c = r[-1]["c"]
            closes = [x["c"] for x in r]
            e = closes[0]
            k2 = 2/(self.cfg.ema_trend+1)
            for v in closes[1:]: e = v*k2 + e*(1-k2)
            side_gap = (c/hi-1)*100 if c>lo else (c/lo-1)*100
            logging.info(f"CORE {sym}: no signal | close {c:.4g} vs channel {
        except Exception:
            pass
    if sig!=0 and sig!=self.state[sym]:
        self._flip(sym, sig, kind, rows, atrv, equity)

```

```

def _flip(self, sym, new_side, kind, rows, atrv, equity):
    for pid, info in list(self.open_pos.items()):
        if info["sym"]==sym:
            if not self.cfg.dry_run:
                p=[x for x in self.client.positions(sym) if x["id"]==pid]
                if p: self.client.place(sym, "SELL" if p[0]["side"]>0 else "B
                self.log(sym,"EXIT","", "", "", "", "", "flip"); del self.open_pos[pid]
    self.state[sym]=0
    if new_side==0: return
    price=rows[-1]["c"]
    rd = atrv/price
    if rd<self.cfg.stop_min or rd>self.cfg.stop_max:
        logging.info(f"{sym}: skipped, ATR stop {rd:.1%} outside band"); retu
    stop = price*(1-rd) if new_side>0 else price*(1+rd)
    target = price*(1+self.cfg.tp_r*rd) if new_side>0 else price*(1-self.cfg.
    risk = self.cfg.risk_map.get(sym, self.cfg.risk_per_trade)
    if self.open_risk() + risk > self.cfg.max_open_risk:
        logging.info(f"{sym}: skipped, open-risk cap reached"); return
    qty = min(risk/rd, self.cfg.max_notional_lev)*equity/price
    qty = max(qty, self.cfg.min_qty_map.get(sym, 0), self.cfg.min_notional_us
    side = "BUY" if new_side>0 else "SELL"
    logging.info(f"{sym}: ENTER {kind} {side} qty={fmt_qty(qty)} stop={stop:.
    d = "🟢 LONG" if new_side>0 else "🔴 SHORT"
    rd = abs(price-stop)/price*100
    rr = abs(target-price)/abs(price-stop)
    notify(f"{d} <b>{sym}</b> · {kind.upper()}\n"
           f"📌 Entry <code>{price:.4f}</code>\n"
           f"🔴 <code>{stop:.4f}</code> (-{rd:.1f}%) 🎯 <code>{target:.4f}<
           f"⚖️ 1 : {rr:.1f} · 🇺🇸 Risk {risk*100:.2f}% · Size ~${qty*price:.(
    if self.cfg.dry_run:
        self.log(sym, kind, side, fmt_qty(qty), f"{stop:.2f}", f"{target:.2f}
    else:
        filled = False
        if self.cfg.entry_limit:
            lim = price*(1-self.cfg.limit_offset) if new_side>0 else price*(1
            _mode, res = self.client.place(sym, side, qty, "OPEN", order_type
                stop=stop, target=target)
            oid = (res or {}).get("orderId") or (res or {}).get("id")
            for _ in range(self.cfg.entry_timeoutBars):
                time.sleep(30)
                self.sync()
                if any(i["sym"]==sym for i in self.open_pos.values()):
                    filled = True; logging.info(f"{sym}: limit filled at make
            if not filled and oid:
                try: self.client.cancel(sym, oid)
                except Exception as ex: logging.warning(f"cancel failed (may

```

```

        logging.info(f"{sym}: limit entry unfilled after {self.cfg.en
        return
    else:
        self.client.place(sym, side, qty, "OPEN", stop=stop, target=target
        time.sleep(1); self.sync()
        filled = any(i["sym"]==sym for i in self.open_pos.values())
        if not filled and not self.cfg.entry_limit:
            logging.error(f"{sym}: entry unconfirmed - not tracked"); return
    for _pid, _info in self.open_pos.items():
        if _info["sym"]==sym and "entry" not in _info:
            _info.update({"entry":price,"stop":stop,"target":target,"side":ne
        self.state[sym]=new_side
        self.entry_info[sym]={"entry":price,"kind":kind,"stop_pct":rd,"stop":stop

def run_scanner(self, equity):
    """Top-gainer cross-sectional momentum sleeve. Validated on two windows
    (crash: 72 tr +22.9%, bull: 87 tr +52.9% at 1.5% risk)."""
    now_ms = time.time()*1000
    win = (int(now_ms)//INTERVAL_MS[self.cfg.interval])*INTERVAL_MS[self.cfg.
    if win == self.scan_seen_bar: return
    self.scan_seen_bar = win
    universe = list(self.cfg.scanner_pairs)
    if self.cfg.scanner_dynamic:
        try:
            tk = self.client.tickers()
            pool = sorted((t for t in tk if t["symbol"].endswith("USDT") and
                           key=lambda t: -t["vol24"]))
            dyn = [t["symbol"] for t in pool[:self.cfg.scanner_max_pairs]]
            if dyn: universe = dyn
            else: logging.warning("dynamic universe empty - using fallback li
        except Exception as e:
            logging.warning(f"ticker fetch failed ({e}) - using fallback list
    data = {}
    live_px = {}
    try:
        live_px = {t["symbol"]: float(t.get("lastPrice") or 0) for t in self.
    except Exception:
        pass
    stale = 0
    for s in universe:
        try:
            try:
                rows = binance_klines(s, self.cfg.interval, 100)
            except Exception as e:
                logging.warning(f"BINANCE FEED DOWN ({e}) - Bitunix fallback
                rows = self.client.klines(s, self.cfg.interval, 100)

```

```

        if len(rows) >= 30:
            rows = rows[:-1] # drop forming bar
            last_t = rows[-1]["t"]
            if time.time()*1000 - last_t > 12*3600e3: # last candle c
                stale += 1
                logging.warning(f"STALE FEED {s}: last candle {last_t} -
                continue
            lp = live_px.get(s)
            scale = 1000.0 if s.startswith("1000") else 1.0 # 1000PEPE
            if lp and abs(rows[-1]["c"]*scale - lp)/lp > 0.02:
                stale += 1
                logging.warning(f"DIVERGED {s}: Binance {rows[-1]['c']} v
                continue
            data[s] = rows
    except Exception:
        continue

if stale:
    notify(f"⚠️ {stale} pairs had stale price data and were excluded (Bit
if not data: return
top = []
for s, r in data.items():
    if len(r) >= 26:
        top.append((r[-1]["c"]/r[-25]["c"]-1, s))
top.sort(reverse=True)
self.last_scan = {"top": [(s, g) for g, s in top[:5]], "universe": len(da
tstr = " · ".join(f"{s.replace('USD','')} {g*100:+.1f}%" for g, s in top
logging.info(f"scanner universe: {len(data)} pairs | top4d: {tstr}")
logging.info(f"SCAN REPORT | universe {len(data)} | top: {tstr} | signals
self.scan_data = data
held = {info["sym"] for info in self.open_pos.values()}
cands = []
for s, r in data.items():
    if s in held: continue
    closes = [x["c"] for x in r]
    ret24 = closes[-1]/closes[-25] - 1
    ret_th = 0.025 if self.cfg.mode == "active" else self.cfg.scanner_ret
    if ret24 > ret_th:
        n = 10 if self.cfg.mode == "active" else 20
        hi20 = max(x["h"] for x in r[-n-1:-1])
        vols = [x["vol"] for x in r[-22:-2]]
        vma = sum(vols)/len(vols) if vols else 0
        brk = closes[-1] > hi20
        vspike = vma > 0 and r[-1]["vol"] > 1.0*vma
        if brk and vspike:
            trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"])), a
                    for i in range(-14, 0)]
            atr = sum(trs)/len(trs)

```

```

        cands.append((ret24, s, r[-1]["c"], atr))
cands.sort(reverse=True)
slots = self.cfg.scanner_max - sum(1 for i in self.open_pos.values() if i
for ret24, s, price, atr in cands:
    if slots <= 0:
        logging.warning(f"GATE-PASS {s} queued but slots full - SKIPPED")
        continue
    rd = atr/price
    if rd < 0.004 or rd > 0.06:
        logging.warning(f"GATE-PASS {s} all signal gates passed but stop
        continue
    stop = price - atr; target = price + self.cfg.tp_r*atr
    qty = (self.cfg.scanner_risk/rd)*equity/price
    qty = min(qty, 5.0*equity/price)
    qty = max(qty, self.cfg.min_qty_map.get(s, 0), self.cfg.min_notional_
    if qty*price < 10:
        logging.warning(f"GATE-PASS {s} all gates passed but size ${qty*p
        continue
    self.last_scan["signals"] += 1
    logging.info(f"SCANNER ENTER {s} (24h +{ret24*100:.1f}%) qty={fmt_qty
    rd = abs(price-stop)/price*100
    notify(f"🚀 <b>SCANNER · {s}</b> 🔥 +{ret24*100:.1f}%/4d\n"
           f"📍 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> (-{
           f"📊 Risk {self.cfg.scanner_risk*100:.1f}%" + foot())
    if self.cfg.dry_run:
        self.log(s, "scanner", "BUY", fmt_qty(qty), f"{stop:.4f}", f"{tar
    else:
        try:
            self.client.place(s, "BUY", qty, "OPEN", stop=stop, target=ta
            time.sleep(1); self.sync()
            for _pid,_info in self.open_pos.items():
                if _info["sym"]==s and "entry" not in _info:
                    _info.update({"entry":price,"stop":stop,"target":targ
            except Exception as e:
                logging.error(f"scanner entry failed {s}: {e}")
        slots -= 1
        held.add(s)
# SHORT engine: weakest movers breaking DOWN with volume (validated: +23R
cands_s = []
for s, r in data.items():
    if s in held: continue
    closes = [x["c"] for x in r]
    ret24 = closes[-1]/closes[-25] - 1
    if ret24 < -(0.025 if self.cfg.mode == "active" else self.cfg.scanner
        n = 10 if self.cfg.mode == "active" else 20
        lo20 = min(x["l"] for x in r[-n-1:-1])
        vols = [x["vol"] for x in r[-22:-2]]

```

```

vma = sum(vols)/len(vols) if vols else 0
brk_dn = closes[-1] < lo20
vspike = vma > 0 and r[-1]["vol"] > 1.0*vma
if brk_dn and vspike:
    trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"])), a
            for i in range(-14, 0)]
    atr = sum(trs)/len(trs)
    cands_s.append((-ret24, s, r[-1]["c"], atr))
cands_s.sort(reverse=True)
slots_s = self.cfg.scanner_max - sum(1 for i in self.open_pos.values() if
for ret24, s, price, atr in cands_s:
    if slots_s <= 0:
        logging.warning(f"GATE-PASS SHORT {s} queued but slots full - SKI
        continue
    rd = atr/price
    if rd < 0.004 or rd > 0.06:
        logging.warning(f"GATE-PASS SHORT {s} stop {rd*100:.2f}% outside
        continue
    stop = price + atr; target = price - self.cfg.tp_r*atr
    qty = (self.cfg.scanner_risk/rd)*equity/price
    qty = min(qty, 5.0*equity/price)
    qty = max(qty, self.cfg.min_qty_map.get(s, 0), self.cfg.min_notional_
    if qty*price < 10:
        logging.warning(f"GATE-PASS {s} all gates passed but size ${qty*p
        continue
    self.last_scan["signals"] += 1
    logging.info(f"SCANNER SHORT {s} (24h -{ret24*100:.1f}%) qty={fmt_qty
    notify(f"📉 <b>SCANNER SHORT · {s}</b> 📉 -{ret24*100:.1f}%/4d\n"
           f"📍 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> · (
           f"📊 Risk {self.cfg.scanner_risk*100:.1f}%" + foot())
    if self.cfg.dry_run:
        self.log(s, "scanner-short", "SELL", fmt_qty(qty), f"{stop:.4f}",
        eq = equity
        self.open_pos[f"scan-{s}-{int(time.time()*1000)}"] = {"sym": s, "
        "side": -1, "entry": price, "stop": stop, "target": target, "
        "risk_frac": self.cfg.scanner_risk, "eq_at_entry": eq}
    else:
        try:
            self.client.place(s, "SELL", qty, "OPEN", stop=stop, target=t
            time.sleep(1); self.sync()
            for _pid,_info in self.open_pos.items():
                if _info["sym"]==s and "entry" not in _info:
                    _info.update({"entry":price,"stop":stop,"target":targ
            except Exception as e:
                logging.error(f"scanner short entry failed {s}: {e}")
        slots_s -= 1
        held.add(s)

```

```

# VOLUME SURGE engine: enter on >=2x volume expansion before the breakout
slots_v = self.cfg.scanner_max - sum(1 for i in self.open_pos.values() if
for s, r in data.items():
    if slots_v <= 0: break
    if s in held: continue
    closes = [x["c"] for x in r]
    vols = [x["vol"] for x in r]
    vma = sum(vols[-21:-1])/20 if len(vols) >= 21 else 0
    if vma <= 0: continue
    vx = vols[-1]/vma
    if vx < self.cfg.surge_mult: continue
    if r[-1]["c"] <= r[-1]["o"]: continue
    k2 = 2/21; e = closes[0]
    for vv in closes[1:]: e = vv*k2 + e*(1-k2)
    if r[-1]["c"] <= e: continue
    price = r[-1]["c"]
    trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"])), abs(r[i]
atr = sum(trs)/len(trs)
rd = atr/price
if not (0.004 <= rd <= 0.06): continue
qty = min((self.cfg.scanner_risk/rd)*equity/price, 5.0*equity/price)
qty = max(qty, self.cfg.min_qty_map.get(s, 0), self.cfg.min_notional_
if qty*price < 10: continue
self.last_scan["signals"] += 1
stop = price-atr; target = price+self.cfg.tp_r*atr
logging.info(f"VOLUME SURGE {s} ({vx:.1f}x vol) entering at {price:.4
notify(f"🔥 <b>VOLUME SURGE · {s}</b>\n{vx:.1f}x volume expansion - e
    f" 📌 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> · (
if self.cfg.dry_run:
    self.log(s, "surge", "BUY", fmt_qty(qty), f"{stop:.4f}", f"{targe
else:
    try:
        tick = 0.0001 if price < 10 else 0.1
        self.client.place(s, "BUY", qty, "OPEN", stop=stop, target=ta
        time.sleep(1); self.sync()
        for _pid, _info in self.open_pos.items():
            if _info["sym"] == s and "entry" not in _info:
                _info.update({"entry": price, "stop": stop, "target":
                    "kind": "scanner", "qty": qty, "risk_fr
                    "eq_at_entry": equity})
        except Exception as ex:
            logging.error(f"surge entry failed {s}: {ex}")
    slots_v -= 1
    held.add(s)
armed_now = 0
for g0, s0 in top[:3]:
    if g0 <= 0: continue

```

```

r = data.get(s0)
if not r: continue
n = 10 if self.cfg.mode == "active" else 20
hi = max(x["h"] for x in r[-n-1:-1])
need = (hi - r[-1]["c"]) / r[-1]["c"]
if 0 < need <= self.cfg.trigger_dist and s0 not in {t["sym"] for t in
    price = r[-1]["c"]
    trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"])), abs(r
    atr = sum(trs)/len(trs)
    rd = atr/price
    if 0.004 <= rd <= 0.06:
        eq = equity
        qty = min((self.cfg.scanner_risk/rd)*eq/price, 5.0*eq/price)
        qty = max(qty, self.cfg.min_qty_map.get(s0, 0), self.cfg.min_
        if qty*price >= 10:
            self.triggers.append({"sym": s0, "side": 1, "level": hi*1
                                "stop": hi*1.001 - atr, "target": h
                                "qty": qty, "expiry": time.time()+8
            armed_now += 1
            notify(f"⚡ <b>TRIGGER ARMED · {s0}</b>\n"
                f"fills if price touches <code>{hi*1.001:.4f}</cod
                f"🔴 <code>{hi*1.001-atr:.4f}</code> · 🎯 <code>{h
# MAJORS always get triggers when near their level - deep books actually
for cs in ("BTCUSDT", "ETHUSDT", "SOLUSDT", "BNBUSDT"):
    if cs not in data or cs in held or cs in {t["sym"] for t in self.trig
    r = data[cs]
    n = 10 if self.cfg.mode == "active" else 20
    hi = max(x["h"] for x in r[-n-1:-1]); lo = min(x["l"] for x in r[-n-1
    c0 = r[-1]["c"]
    for side, lvl, need in ((1, hi*1.001, (hi-c0)/c0), (-1, lo*0.999, (c0
        if 0 < need <= self.cfg.trigger_dist:
            trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"])), a
            atr = sum(trs)/len(trs)
            rd = atr/c0
            if not (0.004 <= rd <= 0.06): continue
            eq = equity
            qty = min((self.cfg.scanner_risk/rd)*eq/c0, 5.0*eq/c0)
            qty = max(qty, self.cfg.min_qty_map.get(cs, 0), self.cfg.min_
            if qty*c0 < 10: continue
            if side > 0:
                self.triggers.append({"sym": cs, "side": 1, "level": lvl,
                                    "target": lvl+self.cfg.tp_r*atr, "q
                nm = f"⚡ <b>TRIGGER ARMED · {cs}</b>\nfills if price tou
            else:
                self.triggers.append({"sym": cs, "side": -1, "level": lvl
                                    "target": lvl-self.cfg.tp_r*atr, "q
                nm = f"⚡ <b>SHORT TRIGGER · {cs}</b>\nfills if price tou

```

```

        armed_now += 1
        logging.info(f"MAJOR TRIGGER ARMED: {cs} side={side} level={l
        notify(nm + foot())
        break

if armed_now:
    logging.info(f"TRIGGERS ARMED: {[t['sym'] for t in self.triggers]}")
detail = ""
n = 10 if self.cfg.mode == "active" else 20
for g0, s0 in top[:3]:
    r = data[s0]
    hi = max(x["h"] for x in r[-n-1:-1])
    lo = min(x["l"] for x in r[-n-1:-1])
    vols = [x["vol"] for x in r[-22:-2]]
    vma = sum(vols)/len(vols) if vols else 0
    vx = r[-1]["vol"]/(vma or 1)
    brk = r[-1]["c"] > hi; brk_dn = r[-1]["c"] < lo
    if brk:
        gate = "LONG READY  "
    elif brk_dn:
        gate = "SHORT READY  "
    else:
        need = (hi - r[-1]['c'])/r[-1]['c']*100 if g0 > 0 else (r[-1]['c'
        gate = f"{need:+.1f}% from trigger · vol {vx:.1f}x"
    detail += f"\n📁 {s0.replace('USDT','')} {g0*100:+.1f}%: {gate}"
for cs in ("BTCUSDT", "ETHUSDT"):
    if cs in data:
        r = data[cs]; nn = 10 if self.cfg.mode == "active" else 20
        hi = max(x["h"] for x in r[-nn-1:-1]); c0 = r[-1]["c"]
        bq = (r[-1]["tb"]/r[-1]["vol"]) if r[-1]["vol"] > 0 else 0.5
        detail += f"\n📁 {cs.replace('USDT','')}: {(c0/hi-1)*100:+.1f}% f
sig = self.last_scan["signals"]
notify(f"🔍 <b>SCAN</b> - {len(data)} pairs swept\n"
f"🏆 {tstr}{detail}\n"
f"{'🚀 ' + str(sig) + ' SIGNAL(S) FIRED' if sig else '✅ all gates

```

```

def poll_commands(self):
    if not (_TG_TOKEN and _TG_CHAT): return
    try:
        r = requests.get(f"https://api.telegram.org/bot{_TG_TOKEN}/getUpdates
                        params={"offset": self.tg_offset, "timeout": 0}, tim
        for u in r.get("result", []):
            self.tg_offset = int(u["update_id"]) + 1
            msg = u.get("message", {})
            if str(msg.get("chat", {}).get("id")) != str(_TG_CHAT): continue
            text = (msg.get("text") or "").strip().lower()
            if text.startswith("/test"):

```

```

        parts = text.split()
        sym = (parts[1].upper() if len(parts) > 1 else "BTCUSDT")
        if not sym.endswith("USDT"): sym += "USDT"
        self.test_trade(sym)
    elif text == "/status": self.status_report()
    elif text in ("/start", "/help"): self.welcome()
    elif text == "/fire": self.fire_trade()
    elif text.startswith("/risk"): self.set_risk(text)
    elif text == "/reset":
        self.risk.trades = []
        notify("🔄 Trade record wiped - 0 trades, clean slate. (Deplo
except Exception:
    pass

```

```

def status_report(self):
    eq = self.equity()
    wins = sum(self.risk.trades); n = len(self.risk.trades)
    wr = f"{wins/n*100:.0f}%" if n else "n/a"
    pos = "\n".join(f"• {i['sym']} (risk {i.get('risk','?')})" for i in self.
    top3 = " · ".join(f"{s.replace('USDT','')} {g*100:+.1f}%" for s, g in sel
    notify(f"📊 <b>STATUS</b>\n"
        f"💰 Equity <b>${eq:.2f}</b> · 📝 {n} trades (WR {wr})\n"
        f"📁 Open: {pos}\n"
        f"🔍 {top3}\n"
        f"🛑 Halted: {self.risk.halted_today or self.risk.killed}" + foot(

```

```

def test_trade(self, sym="BTCUSDT"):
    valid = self.client.valid_symbols()
    if valid and sym not in valid:
        notify(f"⚠️ <b>{sym}</b> is not a listed Bitunix future. Check the Fu
        return
    notify(f"🔪 <b>TEST</b>: firing tiny {sym} order to prove the full pipeli
    try:
        rows = self.client.klines(sym, self.cfg.interval, 50)
        if len(rows) < 20: raise RuntimeError("no kline data")
        price = rows[-1]["c"]
        trs = [max(rows[i]["h"]-rows[i]["l"], abs(rows[i]["h"]-rows[i-1]["c"])
                for i in range(-14, 0)]
        atrv = sum(trs)/len(trs)
        eq = self.equity()
        notional = max(12.0, min(25.0, eq*0.25)) # $12-25 - tiny
        qty = max(notional/price, self.cfg.min_qty_map.get(sym, 0), self.cfg.
        notional = qty*price
        stop = price - 0.6*atrv; target = price + 0.9*atrv
        tick = 0.0001 if price < 10 else 0.1
        self.client.place(sym, "BUY", qty, "OPEN", stop=stop, target=target,
        time.sleep(1); self.sync()

```

```

found = False
for pid, info in self.open_pos.items():
    if info["sym"] == sym and "entry" not in info:
        rf = (abs(price-stop)/price*qty*price)/max(eq,1e-9)
        info.update({"entry": price, "stop": stop, "side": 1, "kind":
logging.info(f"TEST TRADE fired {sym} qty={fmt_qty(qty)} stop={stop:.
notify(f"📈 <b>TEST ENTER {sym}</b>\n"
f"📉 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> · (
f"$💰 ${notional:.0f} (tiny) · ⌚ awaiting resolution..." + foot
except Exception as e:
    logging.error(f"test trade failed: {e}")
    notify(f"📉 TEST FAILED: {e}")

```

```

def manage_exits(self):
    """Close positions with market orders when price hits stop/target.
    This makes exits independent of exchange-side TP/SL bracket validation."""
    for pid, info in list(self.open_pos.items()):
        if not info.get("entry") or not info.get("stop") or not info.get("tar
            continue
        try:
            mk = self.client.ticker_px(info["sym"]) or self.client.klines(inf
        except Exception:
            continue
        # v8 fix: this is the function that actually closes positions, and it
        # on mk before comparing it to stop/target - one bad ticker read (e.g
        # bug above, or any transient garbage value) could close a real posit
        # >25% deviation from entry in a single poll cycle is treated as a ba
        if mk <= 0 or abs(mk / info["entry"] - 1) > 0.25:
            err_once(f"badpx-{info['sym']}", f"{info['sym']}: ticker read {mk
                f"ignoring this cycle, not treating as a stop/target hit
            continue
        side = info.get("side", 1)
        # +1R break-even ratchet: once up 1R, stop moves to entry (Grok audit
        rd1 = abs(info["entry"] - info.get("be_from", info["stop"]))
        if not info.get("be_done") and rd1 > 0:
            if (side > 0 and mk >= info["entry"] + rd1) or (side < 0 and mk <
                info["be_from"] = info["stop"]
                info["stop"] = info["entry"] * (1.001 if side > 0 else 0.999)
                info["be_done"] = True
                logging.info(f"BE-RATCHET {info['sym']}: +1R reached, stop ->
                notify(f"🔒 <b>{info['sym']}</b>: +1R reached - stop moved to
            hit_stop = mk <= info["stop"] if side > 0 else mk >= info["stop"]
            hit_tgt = mk >= info["target"] if side > 0 else mk <= info["target"]
            if not (hit_stop or hit_tgt):
                continue
            exit_p = mk

```

```

R = (exit_p - info["entry"]) / abs(info["entry"] - info["stop"]) * si
try:
    close_side = "SELL" if side > 0 else "BUY"
    try:
        self.client.place(info["sym"], close_side, info["qty"], "CLOS
    except Exception as e1:
        logging.warning(f"close with positionId failed ({e1}) - tryin
        self.client.flash_close(pid)
        logging.info(f"MANAGED EXIT {info['sym']} at {exit_p:.4f} (R={R:+
    except Exception as e:
        logging.error(f"managed exit failed {info['sym']}: {e}")
        continue
self._mr_cool = getattr(self, "_mr_cool", {})
self._mr_cool[info["sym"]] = time.time()
R = max(-3.0, min(3.5, R))
if info.get("kind") != "test":
    self.risk.record_trade(R > 0)
wins = sum(self.risk.trades); n = len(self.risk.trades)
wr = f"{wins}/{n} ({wins/n*100:.0f}%" if n else "0/0"
e2 = "🔪" if info.get("kind")=="test" else ("🟢" if R>0 else "🔴")
w2 = "TEST CLOSED" if info.get("kind")=="test" else ("WIN ✅" if R>0
pct = R * info.get("risk_frac", 0.015) * 100
rf = info.get("risk_frac", 0.015); eq0 = info.get("eq_at_entry", 0)
eq_now = eq0*(1+R*rf) if eq0 else 0
st = streak_of(self.risk.trades)
self.log(info["sym"],"CLOSED","", "", "", "", "", "", f"{R:+.2f}R ({pct:+.2f}%
eqline = f"\n💰 Equity ${eq_now:.2f}" if eq_now else ""
stline = f"🔥 {st} win streak" if st >= 2 else ""
notify(f"{e2} <b>{info['sym']} CLOSED · {w2}</b>\n"
        f"💰 Result: <b>{pct:+.2f}%</b> ({R:+.2f}R)\n🏆 Record: {wr}{s
self.state[info["sym"]] = 0
self.entry_info.pop(info["sym"], None)
del self.open_pos[pid]

```

```

def fire_trade(self):
    """MANUAL OVERRIDE - fires one trade on the hottest coin now. Not a strat
    notify("🔴 <b>MANUAL OVERRIDE</b> - taking the hottest coin now (not a st
    try:
        tk = self.client.tickers()
        hot = sorted((t for t in tk if t.get("symbol","").endswith("USDT") an
                        key=lambda t: float(t.get("lastPrice") or 0)/max(float(t
        sym = None
        for t in hot:
            if t["symbol"] in self.cfg.symbols: continue
            sym = t["symbol"]; break
        if not sym: raise RuntimeError("no suitable coin")
    
```

```

rows = self.client.klines(sym, self.cfg.interval, 100)
if len(rows) < 20: raise RuntimeError("no data")
price = rows[-1]["c"]
trs = [max(rows[i]["h"]-rows[i]["l"], abs(rows[i]["h"]-rows[i-1]["c"])
        for i in range(-14, 0)]
atr = sum(trs)/len(trs)
eq = self.equity()
qty = max(self.cfg.min_notional_usdt/price, 0.001 if "BTC" in sym else
qty = max(qty, (self.cfg.scanner_risk/(atr/price))*eq/price if atr/
qty = min(qty, 5.0*eq/price)
stop = price - atr; target = price + self.cfg.tp_r*atr
tick = 0.0001 if price < 10 else 0.1
self.client.place(sym, "BUY", qty, "OPEN", stop=stop, target=target,
time.sleep(1); self.sync()
for pid, info in self.open_pos.items():
    if info["sym"] == sym and "entry" not in info:
        info.update({"entry":price,"stop":stop,"target":target,"side"
                    "risk_frac":self.cfg.scanner_risk,"eq_at_entry":
logging.info(f"MANUAL OVERRIDE fired {sym}")
notify(f"🔴 <b>MANUAL OVERRIDE ENTER {sym}</b>\n"
f"📌 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> · (
f"⚠️ Manual trade - excluded from the strategy record" + foot(
except Exception as e:
    logging.error(f"manual override failed: {e}")
    notify(f"🔴 OVERRIDE FAILED: {e}")

```

```

def check_triggers(self):
    try:
        b = binance_klines("BTCUSDT", self.cfg.interval, 3)
        if time.time()*1000 - b[-1]["t"] > 12*3600e3:
            logging.warning("STALE BINANCE FEED - trigger fills paused")
            return
    except Exception:
        pass
    """Resting trigger orders: fill when price touches the level (validated +
    if not self.triggers: return
    live_syms = {i["sym"] for i in self.open_pos.values()}
    for t in list(self.triggers):
        if time.time() > t["expiry"] or t["sym"] in live_syms:
            self.triggers.remove(t); continue
    try:
        px = self.client.ticker_px(t["sym"]) or self.client.klines(t["sym
    except Exception:
        continue
    if t["side"] < 0 and px <= t["level"]:
        atr = t["stop"] - t["level"]

```

```

if px <= t["level"] - 1.5*atr:
    logging.warning(f"TRIGGER GAP-SKIP SHORT {t['sym']}")
    notify(f"▶▶ <b>{t['sym']}</b> gapped through the short trigger")
    self.triggers.remove(t); continue
stop = px + 1.0*atr; target = px - self.cfg.tp_r*atr
try:
    tick = 0.0001 if px < 10 else 0.1
    self.client.place(t["sym"], "SELL", t["qty"], "OPEN", stop=stop)
    time.sleep(1); self.sync()
    for pid, info in self.open_pos.items():
        if info["sym"] == t["sym"] and "entry" not in info:
            info.update({"entry": px, "stop": stop, "target": target,
                        "qty": t["qty"], "risk_frac": self.cfg.scanner_risk})
    logging.info(f"TRIGGER FILLED SHORT {t['sym']} at {px:.4f}")
    notify(f"▼ <b>TRIGGER FILLED SHORT · {t['sym']}</b>\n📌 <code>{px:.4f}</code> · 📌 <code>{stop:.4f}</code>")
    self.triggers.remove(t)
except Exception as e:
    logging.error(f"trigger short fill failed {t['sym']}: {e}")
    continue

if t["side"] > 0 and px >= t["level"]:
    atr = t["level"] - t["stop"] # original 1x ATR
    # CHASE PROTECTION: if price already gapped more than halfway to
    if px >= t["level"] + 1.5*atr:
        logging.warning(f"TRIGGER GAP-SKIP {t['sym']}: filled-zone blocked")
        notify(f"▶▶ <b>{t['sym']}</b> gapped through the trigger (+{(t['stop'] - t['level'])/atr:.2f})")
        self.triggers.remove(t); continue
    stop = px - 1.0*atr; target = px + self.cfg.tp_r*atr # RE-ANCHOR
    try:
        tick = 0.0001 if px < 10 else 0.1
        self.client.place(t["sym"], "BUY", t["qty"], "OPEN", stop=stop)
        time.sleep(1); self.sync()
        for pid, info in self.open_pos.items():
            if info["sym"] == t["sym"] and "entry" not in info:
                info.update({"entry": px, "stop": stop, "target": target,
                            "side": 1, "kind": "scanner", "qty": t["qty"],
                            "risk_frac": self.cfg.scanner_risk, "eq_qty": t["qty"]})
        logging.info(f"TRIGGER FILLED {t['sym']} at {px:.4f} stop {stop:.4f}")
        notify(f"🚀 <b>TRIGGER FILLED · {t['sym']}</b>\n📌 <code>{px:.4f}</code> · 📌 <code>{stop:.4f}</code>")
        self.triggers.remove(t)
    except Exception as e:
        logging.error(f"trigger fill failed {t['sym']}: {e}")

def update_dashboard(self):
    """One pinned message, always current - the at-a-glance dashboard."""
    if not (_TG_TOKEN and _TG_CHAT): return
    try:

```

```

eq = self.equity()
wins = sum(self.risk.trades); n = len(self.risk.trades)
wr = f"{wins}/{n} ({wins/n*100:.0f}%)" if n else "awaiting trade #1"
chg = (eq/(self.day_start_eq or eq)-1)*100
st = streak_of(self.risk.trades)
txt = (f"🇺🇸 <b>LEVERAGELORD DASHBOARD</b>\n"
      f"💰 Equity <b>${eq:.2f}</b> ({chg:+.1f}% today)\n"
      f"📁 {len(self.open_pos)} open · 📝 {n} trades · 🏆 {wr}" + (
      f"🎯 Target 5%/mo · verdict at 40 trades\n"
      f"📈 <code>{sparkline(self.eq_hist)}</code>\n"
      f"— live —")
if not getattr(self, "dash_id", None):
    requests.post(f"https://api.telegram.org/bot{_TG_TOKEN}/unpinAllC
                  data={"chat_id": _TG_CHAT}, timeout=10)
    r = requests.post(f"https://api.telegram.org/bot{_TG_TOKEN}/sendM
                      data={"chat_id": _TG_CHAT, "text": txt, "parse_
                            "disable_notification": True}, timeout=10)
    self.dash_id = r.get("result", {}).get("message_id")
    if self.dash_id:
        requests.post(f"https://api.telegram.org/bot{_TG_TOKEN}/pinCh
                      data={"chat_id": _TG_CHAT, "message_id": self.d
                            "disable_notification": True}, timeout=10)
    else:
        requests.post(f"https://api.telegram.org/bot{_TG_TOKEN}/editMessa
                      data={"chat_id": _TG_CHAT, "message_id": self.dash_
                            "parse_mode": "HTML"}, timeout=10)
except Exception:
    pass

```

```

RISK_PROFILES = {
    "low": {"risk_map": {"ETHUSDT": 0.015, "SOLUSDT": 0.015, "BNBUSDT": 0.
                        "DOTUSDT": 0.0075, "NEARUSDT": 0.0075}, "scanner
    "medium": {"risk_map": {"ETHUSDT": 0.025, "SOLUSDT": 0.025, "BNBUSDT": 0.
                        "DOTUSDT": 0.0125, "NEARUSDT": 0.0125}, "scanner
    "high": {"risk_map": {"ETHUSDT": 0.035, "SOLUSDT": 0.035, "BNBUSDT": 0.
                        "DOTUSDT": 0.0175, "NEARUSDT": 0.0175}, "scanner
}

```

```

def set_risk(self, text):
    parts = text.split()
    if len(parts) == 1 or parts[1] in ("status", "show"):
        cur = self.cfg.scanner_risk
        lvl = {0.0075: "LOW", 0.010: "MEDIUM", 0.0125: "HIGH"}.get(cur, f"cus
        notify(f"📌 <b>RISK: {lvl}</b>\n"
              f"Core {self.cfg.risk_map.get('ETHUSDT',0)*100:.1f}% · Scanner
              f"Send /risk low | /risk medium | /risk high to switch" + foot

```

```

        return
    lvl = parts[1].lower()
    if lvl not in self.RISK_PROFILES:
        notify(f"📌 Usage: /risk low · /risk medium · /risk high")
        return
    p = self.RISK_PROFILES[lvl]
    self.cfg.risk_map = dict(p["risk_map"])
    self.cfg.scanner_risk = p["scanner_risk"]
    self.cfg.max_open_risk = p["max_open_risk"]
    logging.info(f"RISK switched to {lvl.upper()} via Telegram")
    notify(f"📌 <b>RISK → {lvl.upper()}</b>\n"
           f"Core {p['risk_map']['ETHUSD']*100:.1f}% · Scanner {p['scanner_risk']}% · Max Open {p['max_open_risk']}%
           f"Applies to all NEW trades. Note: resets to config default on red")

def welcome(self):
    notify(f"👋 <b>Welcome to LeverageLord</b> – your personal trading bot.\n"
           f"📌 The <b>pinned dashboard</b> (top of this chat) shows live equity\n"
           f"⚡ <b>The bot trades automatically</b> – breakout triggers, volume\n"
           f"with stops and targets managed for you. Your job: pick a risk level\n"
           f"📌 /risk low | medium | high – adjust risk appetite\n"
           f"📈 /status – live report anytime\n"
           f"🔪 /test – fire a tiny test trade (proves the pipeline)\n"
           f"🔄 /reset – wipe the trade record\n\n"
           f"⚠️ <b>Trading involves real risk.</b> Start small, never risk what you can't afford\n"
           f"and let the stops do their job." + foot())

def selfcheck(self):
    lines = []
    for sym in ("ETHUSD", "SOLUSD"):
        try:
            rows = binance_klines(sym, self.cfg.interval, 30)
            has_tb = rows[-1].get("tb", -1) >= 0
            lines.append(f"{sym}: {len(rows)} bars ({'binance' if has_tb else 'no trade'})")
        except Exception as e:
            lines.append(f"{sym}: NO FEED ({e})")
    try: lines.append(f"account equity: ${self.equity():.2f}")
    except Exception as e: lines.append(f"account: FAIL ({e})")
    lines.append(f"mode {self.cfg.mode} · poll {self.cfg.poll_seconds}s")
    notify("<b>SELF-CHECK</b>\n" + "\n".join(lines))

def check_intrabar_surge(self):
    """Mid-bar volume-shift entries: forming candle's volume pace vs average,
    Fires on the volume increase the user asked for – no waiting for the 4h candle
    if not self.scan_data: return
    now = time.time()

```

```

self._mr_cool = getattr(self, "_mr_cool", {})
for s, t in list(self._mr_cool.items()):
    if now - t > 7200: del self._mr_cool[s]
if self.open_risk() + self.cfg.scanner_risk > self.cfg.max_open_risk: ret
held = {i["sym"] for i in self.open_pos.values()} | {t["sym"] for t in se
watch = [s for s, g in self.last_scan.get("top", [])[:4] if g > 0] + ["BT
iv = INTERVAL_MS.get(self.cfg.interval, 14400e3)
for s in dict.fromkeys(watch):
    if s in held or s not in self.scan_data: continue
    if now - self._mr_cool.get(s, 0) < 600: continue # 30min per-symbol
    r = self.scan_data[s]
    try:
        cur = binance_klines(s, self.cfg.interval, 1) # forming bar
        bar = cur[-1]
    except Exception:
        continue
    elapsed = min(max((time.time()*1000 - bar["t"]) / iv, 0.05), 1.0)
    vols = [x["vol"] for x in r[-21:-1]]
    vma = sum(vols)/len(vols) if vols else 0
    if vma <= 0: continue
    pace = bar["vol"] / (elapsed * vma)
    if pace < 1.5: continue # volume running
    if bar["c"] <= bar["o"]: continue # must be a gree
    closes = [x["c"] for x in r]
    k2 = 2/21; e = closes[0]
    for v in closes[1:]: e = v*k2 + e*(1-k2)
    if bar["c"] <= e: continue # above short-te
    price = bar["c"]
    trs = [max(r[i]["h"]-r[i]["l"], abs(r[i]["h"]-r[i-1]["c"]), abs(r[i]
    atr = sum(trs)/len(trs)
    rd = atr/price
    if not (0.004 <= rd <= 0.06): continue
    eq = self.equity()
    qty = min((self.cfg.scanner_risk/rd)*eq/price, 5.0*eq/price)
    qty = max(qty, self.cfg.min_qty_map.get(s, 0), self.cfg.min_notional_
    if qty*price < 10: continue
    stop = price - atr; target = price + self.cfg.tp_r*atr
    self.last_scan["signals"] += 1
    logging.info(f"INTRABAR SURGE {s} pace {pace:.1f}x entering at {price
    notify(f"🔥 <b>INTRABAR SURGE · {s}</b>\n{pace:.1f}x volume pace mid-
        f"📌 <code>{price:.4f}</code> · 🛑 <code>{stop:.4f}</code> · (
    try:
        tick = 0.0001 if price < 10 else 0.1
        self.client.place(s, "BUY", qty, "OPEN", stop=stop, target=target
        time.sleep(1); self.sync()
        for _pid, _info in self.open_pos.items():
            if _info["sym"] == s and "entry" not in _info:

```

```

        _info.update({"entry": price, "stop": stop, "target": tar
                      "kind": "scanner", "qty": qty, "risk_frac":
                      "eq_at_entry": eq})

        held.add(s)
        self._mr_cool[s] = now
    except Exception as ex:
        logging.error(f"intrabar surge entry failed {s}: {ex}")
        break    # one intrabar entry per poll cycle max

def loop(self):
    logging.info(f"Bot v5 starting. dry_run={self.cfg.dry_run}")
    self.selfcheck()
    valid = None if self.cfg.dry_run else self.client.valid_symbols()
    if valid:
        bad = [s for s in self.cfg.symbols if s not in valid]
        if bad:
            logging.warning(f"NOT LISTED on Bitunix futures, removing: {bad}")
            notify(f"⚠️ Not listed on Bitunix futures, removed: {'', ' '.join(ba
            self.cfg.symbols = [s for s in self.cfg.symbols if s in valid]
        bad_sc = [s for s in self.cfg.scanner_pairs if s not in valid]
        if bad_sc:
            logging.warning(f"scanner pairs not listed, removing: {bad_sc}")
            self.cfg.scanner_pairs = [s for s in self.cfg.scanner_pairs if s
            self.state = {s: 0 for s in self.cfg.symbols}
    mode = "🔴 LIVE (real money)" if not self.cfg.dry_run else "🟡 DRY-RUN (r
    notify(f"🤖 <b>LEVERAGELORD ONLINE</b>\n"
           f"{mode} · MODE: {self.cfg.mode.upper()} · {len(self.cfg.symbols)}
           f"🎯 Target ~5%/mo · verdict at 40 trades\n"
           f"🛡️ Breaker -5%/day · kill switch armed" + foot())
    beats = 0
    beats_between = max(1, 3600 // max(self.cfg.poll_seconds, 1))    # ~1 per
    while True:
        try:
            if not self.cfg.dry_run:
                for eng, fn in (("sync", self.sync), ("triggers", self.check_
                               ("intrabar", self.check_intrabar_surge), ("ex
                try: fn()
                except Exception as ex: err_once(f"eng-{eng}", f"{eng} en
            eq=self.equity()
            if eq<=0:
                if not getattr(self, "_warned_eq", False):
                    self._warned_eq=True
                    logging.error("equity read as 0 - API parse problem, skip
                    time.sleep(self.cfg.poll_seconds); continue
            if self.risk.daily_loss_hit(eq) or self.risk.kill_switch():
                time.sleep(self.cfg.poll_seconds); continue
            for sym in self.cfg.symbols:

```

```

        try: self.run_symbol(sym, eq)
        except Exception as ex: err_once(f"sym-{sym}", f"{sym} cycle")
    try: self.run_scanner(eq)
    except Exception as ex: err_once("scanner", f"scanner error: {ex}")
    beats += 1
    if beats >= beats_between:
        beats = 0
        wins = sum(self.risk.trades); n = len(self.risk.trades)
        wr = f"{wins/n*100:.0f}%" if n else "n/a"
        self.eq_hist.append(eq); self.eq_hist = self.eq_hist[-168:]
        today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
        if today != self.day_stamp:
            base = self.day_start_eq or eq
            self.day_start_eq = eq
            self.day_stamp = today
            chg = (eq/base-1)*100 if base else 0
            c = "🟢" if chg>=0 else "🔴"
            notify(f"📅17 <b>DAILY SUMMARY · {today}</b>\n"
                f"💰 Equity <b>${eq:.2f}</b> ({c}{chg:+.1f}% today"
                f"📝 {n} trades · 🏆 WR {wr}" + (f" · 🔥{streak_of}"
                f"📈 <code>{sparkline(self.eq_hist)}</code>" + foot()
            self.update_dashboard()
            status = "✅ running" if not (self.risk.halted_today or self.
            logging.info(f"heartbeat: equity ${eq:.2f} open={len(self.ope
            top3 = " · ".join(f"{s.replace('USDT','')} {g*100:+.1f}%" for
            st = streak_of(self.risk.trades)
            chg = (eq/(self.day_start_eq or eq)-1)*100
            notify(f"💖 <b>Equity ${eq:.2f}</b> ({chg:+.1f}% today) · {st
                f"📁 {len(self.open_pos)} open · 📝 {n} trades · 🏆 {1
                f"🔍 {top3}\n"
                f"📈 <code>{sparkline(self.eq_hist)}</code>" + foot())
            self.poll_commands()
            time.sleep(self.cfg.poll_seconds)
    except Exception as ex:
        logging.error(f"loop error: {ex}", exc_info=True)
        self._consec = getattr(self, "_consec", 0) + 1
        if self._consec >= 5:
            notify(f"💀 <b>BOT CRASH-RESTARTING</b>\n5 consecutive loop er
            logging.error("crash-restart: 5 consecutive loop errors"); os
            time.sleep(60)
    else:
        self._consec = 0

```

```

# ===== ACCESS BOT (funnel delivery) =====
# Runs as a thread if ACCESS_TOKEN is set. Users message the access bot,
# submit their Bitunix UID, you verify in the partners portal and reply

```

/approve <chat_id> - the bot sends them the file + guide automatically.

ACCESS_GUIDE = ""🤖 Your LeverageLord setup (20 minutes):

- 0 Download the bot file: <https://raw.githubusercontent.com/kwalmsley98/LeverageLord/master/bot.py>
- 1 Bitunix: create API keys (Profile > API Management) - enable TRADE only, never
- 2 Telegram: message @BotFather > /newbot > save the token
- 3 Telegram: message @userinfobot > save your chat ID
- 4 Railway.app: sign up > New Project > Deploy from GitHub > upload the bot file
- 5 Railway > Variables: add BITUNIX_API_KEY, BITUNIX_API_SECRET, TG_TOKEN, TG_CHAT_ID
- 6 Deploy > wait for "LeverageLord is alive" in your Telegram

Start with DRY_RUN=true for a day to watch it, then go live small.

Commands once live: /start /status /risk /test /reset /fire

Never share your API secret with anyone. Trading involves risk - start small."""

```
def access_bot_thread():
    tok = os.getenv("ACCESS_TOKEN", "")
    if not tok:
        return
    offset = 0
    pending = {}
    time.sleep(5)
    while True:
        try:
            r = requests.get(f"https://api.telegram.org/bot{tok}/getUpdates",
                             params={"offset": offset, "timeout": 25}, timeout=35)
            for u in r.get("result", []):
                offset = u["update_id"] + 1
                msg = u.get("message", {})
                cid = msg.get("chat", {}).get("id")
                text = (msg.get("text") or "").strip()
                if not cid: continue
                if text.startswith("/start"):
                    requests.post(f"https://api.telegram.org/bot{tok}/sendMessage",
                                  data={"chat_id": cid, "text": "🤖 LeverageLord access bot"})
                elif text.replace(" ", "").isdigit() and 4 <= len(text.replace(" ", "")):
                    uid = text.replace(" ", "")
                    pending[cid] = uid
                    requests.post(f"https://api.telegram.org/bot{tok}/sendMessage",
                                  data={"chat_id": int(_TG_CHAT or 0), "text": f"🔔 ACCESS granted to {cid}"})
                    requests.post(f"https://api.telegram.org/bot{tok}/sendMessage",
                                  data={"chat_id": cid, "text": "✅ UID received. Verification in progress"})
                elif text.startswith("/approve") and str(cid) == str(_TG_CHAT):
                    try:
                        target = int(text.split()[1])
                        uid = pending.get(target, "verified")
                        requests.post(f"https://api.telegram.org/bot{tok}/sendMessage",
                                      data={"chat_id": cid, "text": f"✅ Approved {uid}"})
                    except:
                        requests.post(f"https://api.telegram.org/bot{tok}/sendMessage",
                                      data={"chat_id": cid, "text": "❌ Invalid target"})
                else:
                    continue
```

```

        data={"chat_id": target, "text": f"🎉 Approved! Here
requests.post(f"https://api.telegram.org/bot{tok}/sendMes
        data={"chat_id": int(_TG_CHAT), "text": f"✅ Approved
del pending[target]
except Exception as e:
    requests.post(f"https://api.telegram.org/bot{tok}/sendMes
        data={"chat_id": int(_TG_CHAT), "text": f"approve fai
except Exception:
    time.sleep(5)
time.sleep(1)

if __name__ == "__main__":
    threading.Thread(target=access_bot_thread, daemon=True).start()

if __name__ == "__main__":
    cfg = Config(dry_run=os.getenv("DRY_RUN", "true").lower()!="false")
    client = BitunixClient(os.environ["BITUNIX_API_KEY"], os.environ["BITUNIX_API
    Bot(cfg, client).loop()

```